

HOW TO ANSWER SCENARIO-BASED TECHNICAL INTERVIEWS

Real
Scenarios
Real Skills
Real Growth

A PRACTICAL GUIDE FOR DEVOPS, CLOUD, SRE,
PLATFORM AND INFRASTRUCTURE ENGINEERS



THINK
CLEARLY



TROUBLESHOOT
METHODICALLY



COMMUNICATE
CONFIDENTLY



SOLVE
REAL PROBLEMS



PASS
INTERVIEWS



- ✓ DEVOPS
- ✓ CLOUD
- ✓ SRE
- ✓ PLATFORM
- ✓ INFRASTRUCTURE



Good
Engineers
Explain
Their Thinking

- ✓ LINUX
- ✓ KUBERNETES
- ✓ NETWORKING
- ✓ CI/CD
- ✓ MONITORING
- ✓ AND MORE

SCENARIOS BUILD BETTER ENGINEERS

WHAT SCENARIO-BASED INTERVIEWS ARE REALLY TESTING

It's not just what you know, but how you think, solve, and communicate in real situations.

1 KNOWLEDGE QUESTIONS vs SCENARIO QUESTIONS

Knowledge Questions

- Test facts and definitions
- Usually have one correct answer
- Example: What is a Pod in Kubernetes?

Scenario Questions

- Test real-world thinking
- Have multiple valid approaches
- Example: "Your application is down. How do you investigate and fix it?"

2 WHY INTERVIEWERS USE PRODUCTION SCENARIOS



- Real environments are complex
- There is rarely one perfect answer
- They want to see how you think
- It shows your readiness for real engineering work
- It helps them judge your problem-solving and communication skills

3 TECHNICAL KNOWLEDGE vs ENGINEERING JUDGMENT



Technical Knowledge

- Tools, commands, concepts
- Important, but not enough on its own



Engineering Judgment

- Knowing what to do, when, and why
- Making safe, reasonable decisions with limited information

4 TROUBLESHOOTING ABILITY



- Breaking down a problem
- Gathering the right evidence
- Forming and testing hypotheses
- Identifying the root cause
- Explaining your thought process

5 COMMUNICATION UNDER UNCERTAINTY



- Clearly explaining your thinking
- Asking the right questions
- Keeping stakeholders informed
- Staying calm under pressure
- Admitting what you do not know

6 RISK AWARENESS



- Understanding the impact of changes
- Avoiding unsafe or destructive actions
- Considering security, data, and availability
- Choosing the safest approach

7 KNOWING WHEN TO ESCALATE



- Not every problem is yours to solve alone
- Escalate when there is high impact, risk, or lack of expertise
- Show that you understand your limits
- Explain who you would escalate to and why

8 WHY REASONING CAN MATTER MORE THAN THE FINAL ANSWER



- Interviewers want to see your thinking
- A clear, structured approach is valued
- There can be multiple valid solutions
- Good reasoning shows you can learn, adapt, and work in real engineering teams

★ JUNIOR ENGINEER TIP



Do not rush straight to a fix.

Take time to understand the problem, ask questions, gather evidence, think it through, and explain your approach.

Think
Before
You Fix



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

DEVOPS | CLOUD | SRE | PLATFORM | INFRASTRUCTURE

Learn
Build
Grow
with VERIQTA

THE FRAMEWORK FOR ANSWERING ANY TECHNICAL SCENARIO

A clear, structured approach to analyze problems, find the right solution, and communicate like a professional engineer.

1 THE 8-STEP FRAMEWORK



2 WHAT EACH STEP MEANS

1. CLARIFY	2. ASSESS IMPACT	3. GATHER EVIDENCE	4. HYPOTHEZIZE	5. INVESTIGATE	6. FIX	7. VERIFY	8. PREVENT
- Understand what is actually happening - Ask questions - Confirm assumptions - Identify missing information	- Determine scope and severity - Who is affected? - Is it production? - Is there a workaround? - Any data or security risk?	- Check logs - Review metrics - Look at events - Check configuration - Inspect system state - Find out what changed	- List possible causes - Use your knowledge and evidence - Prioritize the most likely causes	- Test your hypotheses - Use the right tools/commands - Eliminate possibilities - Follow the evidence	- Apply the safest fix - Consider short-term and long-term impact - Make minimal, controlled changes	- Confirm the issue is resolved - Check application health - Ensure no new issues	- Discuss how to avoid it in the future - Suggest monitoring, alerts, or automation - Share lessons learned

3 MINI SCENARIO: APPLICATION IS DOWN AFTER A DEPLOYMENT

Scenario: A web application is returning 500 errors after a new deployment. Users cannot access the service.

Step	Example Answer (What You Might Say)	Key Points
1 Clarify	• I would confirm what errors users are seeing, when it started, whether it affects all users, and what changed recently.	Ask the right questions. Confirm assumptions.
2 Assess Impact	• It seems to be affecting all users in production, so this is a high severity issue.	Identify scope, severity, and business impact.
3 Gather Evidence	• I would check application logs, Kubernetes events, pod status, recent deployment changes, and monitoring metrics.	Use multiple sources of evidence.
4 Hypothesize	• Possible causes could be a bad image, misconfiguration, failed migration, or missing environment variable.	List and prioritize likely causes.
5 Investigate	• I would inspect pod logs, describe the deployment, compare the new and old configuration, and check the image version.	Test your hypotheses, don't guess.
6 Fix	• If the new image is causing the issue, I would roll back to the previous version to restore service.	Apply the safest and fastest fix.
7 Verify	• I would confirm the application is responding, check metrics, and ensure users can access it again.	Verify the fix actually worked.
8 Prevent	• I would suggest better testing, automated checks, and a staged deployment process to catch issues earlier.	Explain how to reduce the chance of recurrence.



JUNIOR ENGINEER TIP

Do not rush straight to a fix. Take time to understand the problem, think it through, and explain your reasoning. Interviewers value your thinking process.

Good
Engineers
Ask Better
Questions



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

LEARN | PRACTICE | GROW | BUILD | SUCCEED WITH VERIQTA

Tech
Skills
Real
Careers

HOW TO CLARIFY AN AMBIGUOUS INTERVIEW QUESTION

Ask the right questions, make reasonable assumptions, and show your thinking.

1 WHY SCENARIOS INTENTIONALLY OMIT INFORMATION



- Real production problems are often unclear and incomplete.
- Interviewers want to see how you handle uncertainty.
- It tests your critical thinking, not just technical knowledge.
- You are expected to ask questions and make reasonable assumptions.

In real engineering work, you rarely get a perfect description of the problem. You have to clarify, investigate, and make informed decisions.

2 NEVER INVENT MISSING FACTS



- Do not assume things that are not stated.
- Do not guess just to move the conversation forward.
- If you are unsure, say what you need to know.
- Making up facts can lead to the wrong solution.
- It is better to ask questions than to give an incorrect answer with high confidence.

"I don't have enough information yet. I would like to confirm a few details before proceeding."

3 ASK TARGETED QUESTIONS



- Ask clear and relevant questions.
- Focus on information that affects your troubleshooting approach.
- Show that you understand what matters.
- Avoid asking unnecessary questions.
- Prioritize questions based on impact and urgency.

5 HOW TO STATE ASSUMPTIONS



- If the interviewer cannot provide more information, clearly state your assumptions.
- Explain why you are making each assumption.
- Confirm that you will proceed based on those assumptions.
- Be ready to adjust if new information becomes available.

Example:

"Since you mentioned users cannot access the application, I will assume this is a production environment and that the issue affects all users. I will proceed based on this assumption. Let me know if that's incorrect."

4 KEY QUESTIONS TO ASK

- 1  Production or development?
- 2  One host or many?
- 3  One user or everyone?
- 4  When did it start?
- 5  What changed recently?
- 6  Is the application completely unavailable or degraded?
- 7  What evidence is available?
- 8  Anything else I should know (such as recent deployments, maintenance, or related systems)?

6 JUNIOR ENGINEER TIP



Do not rush straight to a fix!

Take time to clarify the situation, ask questions, and understand the full context. This shows maturity and real engineering mindset.

Good
Questions
Lead to
Better
Solutions



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

START WITH IMPACT, SCOPE, AND SEVERITY

Before jumping into tools or fixes, understand the impact. This helps you prioritize, communicate effectively, and choose the right response.

1 WHAT IS AFFECTED?



- Which systems, services, or components are affected?
- Is it one server, one pod, a whole environment, or a customer-facing service?
- What features are not working?

2 WHO IS AFFECTED?



- Internal users only?
- External customers?
- A specific team or organization?
- All users or just a small group?

3 HOW WIDESPREAD IS IT?



- Is it isolated to one instance or region?
- Is it happening across multiple hosts, clusters, or regions?
- Is it a single service or a whole platform?

4 IS PRODUCTION DOWN?



- Is this a production issue or a non-production environment?
- Is the service completely down or partially working?
- Are users unable to access it at all?

5 IS DATA AT RISK?



- Is there a risk of data loss or corruption?
- Are writes failing?
- Is there any chance of inconsistent data?
- Do you need to stop certain operations?

6 IS SECURITY INVOLVED?



- Could this be a security incident?
- Is there unauthorized access?
- Are credentials, secrets, or sensitive data exposed?
- Do you need to follow an incident response process?

7 IS THERE A WORKAROUND?



- Can users still achieve their goal another way?
- Is there a temporary fix or manual process?
- Does the business need an immediate solution or can it wait?

8 DEGRADATION vs OUTAGE



- **Service degradation:** Some features are slow or not working, but the service is still usable.
- **Complete outage:** The service is entirely unavailable.

9 WHY SEVERITY CHANGES YOUR RESPONSE



- Severity determines urgency, resources, and communication.
- Critical issues require faster action and escalation.
- Lower-severity issues may allow for more investigation before making changes.

10 EXAMPLE: ONE UNHEALTHY POD vs ENTIRE CUSTOMER-FACING SERVICE

One Unhealthy Pod



- Affects a single instance.
- Service may still be available.
- Lower severity.
- You can investigate and fix with minimal impact.
- Example: One pod is in CrashLoopBackOff.

Entire Customer-Facing Service



- Affects all or most users.
- Business impact can be high.
- Requires immediate attention and escalation.
- You may need to roll back, an incident response.
- Example: The main web application is down and customers cannot access it.

11 KEY TAKEAWAYS

- ✓ Always assess impact before taking action. Understand the scope and who is affected.
- ✓ Check for data and security risks.
- ✓ Look for a workaround if available.
- ✓ Distinguish between service degradation and complete outage.
- ✓ Let severity guide your response.
- ✓ Clear impact assessment shows maturity as an engineer.

12 JUNIOR ENGINEER TIP



Do not rush straight to a fix.

Take a few minutes to understand the impact, scope, and severity. This helps you make the right decision and avoid making the situation worse.

Think
Assess
Prioritize
Then Act



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

THINK IN EVIDENCE, NOT GUESSES

Good engineers use evidence to find the real cause, not guess and hope. Gather information, think critically, and make informed decisions.

1 SYMPTOMS vs CAUSES

Symptoms

- What you see
- What you experience
- Tells you something is wrong
- Not always the real problem

Causes

- The actual reason
- May not be obvious
- Can have multiple possible causes
- You need evidence to confirm



One symptom can have many causes.
Always investigate before assuming.

3 WHY "I WOULD RESTART IT" IS USUALLY A WEAK FIRST ANSWER



- Does not show investigation
- May hide the real problem
- Can cause more issues
- Not safe in production
- Shows a guess, not engineering thinking

"A restart might be part of the solution, but it should come after investigation, not before."

4 STRONG INTERVIEW LANGUAGE



- "I would first verify the symptoms..."
- "I would check the logs to understand what is happening..."
- "I would look at the metrics to see if there are any anomalies..."
- "I would review recent changes..."
- "I would confirm the scope and impact before taking action..."

2 KEY SOURCES OF EVIDENCE

	Logs	Application logs, system logs, container logs (e.g. <code>kubectl logs</code> , <code>journalctl</code>)
	Metrics	CPU, memory, disk, network, application metrics (e.g. Prometheus, CloudWatch)
	Traces	Request flow across services (e.g. Jaeger, OpenTelemetry, X-Ray)
	Events	System and Kubernetes events (e.g. <code>kubectl get events</code>)
	Configuration	Environment variables, config files, Kubernetes manifests, cloud settings
	Process state	Running processes, service status (e.g. <code>ps</code> , <code>systemctl</code> , <code>top</code>)
	Network state	Connectivity, DNS, routing, firewall rules (e.g. <code>ping</code> , <code>curl</code> , <code>traceroute</code> , <code>netstat</code>)
	Deployment history	Recent deployments, version changes, rollbacks (e.g. <code>kubectl rollout history</code>)
	Recent changes	Code changes, configuration updates, infrastructure changes, maintenance

5 EXAMPLE: THINKING IN EVIDENCE

Scenario: Users report the application is slow.

1	Check symptoms	Confirm what is slow and who is affected.
2	Gather evidence	Check logs, metrics, traces, and recent changes.
3	Analyze	Look for patterns and possible causes.
4	Test	Validate your hypothesis with evidence.
5	Fix	Apply the safest and most appropriate fix.
6	Verify	Confirm the issue is resolved.

6 JUNIOR ENGINEER TIP



Do not jump to a fix.

Take time to gather evidence, understand the real cause, and explain your thinking. This shows maturity and builds trust as an engineer.

Better
Engineers
Find the Why



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

BUILD AND TEST HYPOTHESES

A structured way to go from symptoms to a confirmed cause. Good engineers test one possibility at a time, use evidence, and adjust their hypothesis as they learn more.

1 WHAT IS A TROUBLESHOOTING HYPOTHESIS?



- A possible explanation for the problem.
- Not a guess, but an educated idea based on symptoms and knowledge.
- Helps you decide what to check next.
- It can change as you find more evidence.

"A hypothesis is a starting point, not a final answer."

2 TURNING SYMPTOMS INTO POSSIBLE CAUSES



- Start with what you see (symptoms).
- List all possible causes.
- Use your knowledge and experience.
- Consider multiple layers (infrastructure, network, application, etc.).

"One symptom can have many causes. Always think broadly."

3 RANKING LIKELY CAUSES



Prioritize based on:

- How common the issue is
- Recent changes
- Past experience
- Evidence you already have
- Eliminate unlikely causes early

"Focus on the most likely causes first, but keep an open mind."

4 TESTING ONE ASSUMPTION AT A TIME



- Test your hypothesis with specific checks or commands.
- Look for evidence that confirms or disproves it.
- Change only one variable at a time.
- Move to the next hypothesis based on the result.

"Test, don't guess."

5 AVOIDING RANDOM COMMANDS



- Running random commands wastes time.
- It can create confusion or make things worse.
- Be intentional about what you check and why.
- Use a logical and structured approach.

"Every command should have a purpose."

6 CORRELATION vs CAUSATION



- Just because two things happen at the same time does not mean one caused the other.
- Look for real evidence, not coincidences.
- Confirm the actual cause before taking action.

"Things can be related without one causing the other."

7 ELIMINATING POSSIBILITIES



- Use evidence to rule out causes.
- If a test shows something is working, move to the next possibility.
- Keep narrowing down the list until you find the real cause.
- Document what you have eliminated.

"Every test gives you an answer, even if it's not the cause."

8 UPDATING YOUR HYPOTHESIS WHEN EVIDENCE CHANGES



- Be ready to adjust when new evidence appears.
- Remove causes that are no longer likely.
- Add new possibilities if needed.
- Keep an open and flexible mindset.

"Good engineers adapt their thinking based on evidence."

9 EXAMPLE: NARROWING THE FAILURE DOMAIN

Application unavailable

DNS?
Network?
Load balancer?
Application?
Database?

Possible causes

Gather evidence

Narrow the failure domain

10 KEY TAKEAWAYS



- Start with symptoms, not solutions.
- Form and test hypotheses.
- Use evidence, not guesses.
- Eliminate possibilities.
- Understand the difference between correlation and causation.
- Be ready to update your hypothesis.
- A structured approach saves time and leads to the right solution.

★ JUNIOR ENGINEER TIP

Do not jump to a fix.
Take time to build and test your hypothesis. This shows critical thinking and real engineering skills.

Think
Investigate
Learn
Improve

Build
Your
Future
with VERIQTA



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

ANSWERING LINUX AND SERVER FAILURE SCENARIOS

Understand the problem, use the right tools, find evidence, and explain your reasoning.
It's not about memorizing commands, but knowing what they tell you.

1 COMMON SCENARIO EXAMPLES

-  **Server is suddenly slow**
(High response time, slow commands)
-  **Disk is full**
(No space left on device)
-  **CPU is at 100%**
(High CPU usage, server not responding)
-  **Memory usage is high**
(Low memory, swapping, OOM issues)
-  **Service will not start**
(Service fails or keeps stopping)
-  **Permission denied**
(Cannot access file, directory, or command)
-  **Process keeps dying**
(Application or service crashes repeatedly)

2 REASONING APPROACH

- 1 Understand the symptom**
What exactly is happening?
- 2 Check the impact**
Is it one service or the whole server?
- 3 Gather evidence**
Use the right commands and logs.
- 4 Find possible causes**
Based on the evidence.
- 5 Test each possibility**
One at a time.
- 6 Identify the root cause**
Confirm with evidence.
- 7 Apply the appropriate fix**
Use the safest approach.
- 8 Verify and monitor**
Make sure the issue is resolved.

3 KEY MINDSET



- Do not guess.
- Do not run random commands.
- Use evidence.
- Think step by step.
- Understand what the command tells you.
- Explain your reasoning.
- Focus on the problem, not memorization.

"The goal is to show how you think, not just what you know."

4 COMMON LOG LOCATIONS



- /var/log/ (system logs)
- /var/log/syslog
- /var/log/messages
- journalctl (systemd logs)
- Application specific logs (e.g. /var/log/nginx/, /var/log/httpd/)

5 USEFUL LINUX COMMANDS FOR TROUBLESHOOTING

Command	What It Does	What It Helps You Find (Proves)	Example Usage
<code>uptime</code>	Shows system uptime and load average	How long the server has been running, current load	<code>uptime</code>
<code>top</code>	Shows real-time running processes	Which processes are using CPU and memory	<code>top</code>
<code>free</code>	Shows memory usage	Total, used, free memory and swap	<code>free -h</code>
<code>df</code>	Shows disk space usage	Which filesystems are full	<code>df -h</code>
<code>du</code>	Shows directory size	Which directories are using the most space	<code>du -sh /var/log</code>
<code>ps</code>	Shows running processes	Whether a process is running	<code>ps aux grep <process></code>
<code>systemctl</code>	Manages systemd services	Service status, start, stop, restart, failures	<code>systemctl status <service></code>
<code>journalctl</code>	Views system logs	Detailed logs for services and system events	<code>journalctl -u <service> -n 100</code>

6 REMEMBER



- Each command gives you evidence.
- Combine multiple sources of information.
- Always verify the cause before applying a fix.
- Explain what you see and what it means.
- This approach works in real jobs and in interviews.

7 JUNIOR ENGINEER TIP



Do not rush to a fix.

Take time to gather evidence, understand the cause, and explain your reasoning. A restart might hide the problem instead of solving it.

Investigate
Understand
Solve
Grow

Better
Engineers
A Brighter
Future

ANSWERING NETWORKING AND CONNECTIVITY SCENARIOS

A step-by-step approach to diagnose and solve real network and connectivity problems in technical interviews and in production.

1 COMMON SCENARIOS

- Application cannot reach another service
- Connection refused
- Connection timeout
- DNS resolution failure
- Wrong port
- Firewall / security rule
- Routing problem
- Service listening only on localhost



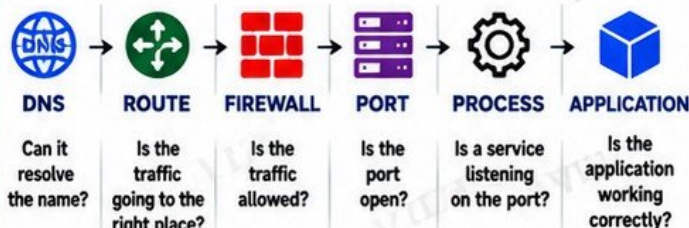
REAL INTERVIEW EXAMPLES

"My application cannot connect to the database. How would you troubleshoot it?"

"A service is timing out when calling another service. What would you check?"

2 INVESTIGATION MODEL

Follow a logical order to narrow down the issue.



Check each layer in order.
Do not jump to conclusions.

3 USEFUL TOOLS AND COMMANDS

Tool / Command	What It Does	Example Usage
<code>dig</code>	DNS lookup	<code>dig example.com</code>
<code>nslookup</code>	DNS lookup	<code>nslookup example.com</code>
<code>curl</code>	Test HTTP/HTTPS connectivity	<code>curl -v http://example.com</code>
<code>ping</code>	Test network connectivity (where appropriate)	<code>ping example.com</code>
<code>traceroute</code>	Show network path	<code>traceroute example.com</code>
<code>ss</code>	Check listening ports	<code>ss -tln</code>
<code>nc</code>	Test port connectivity	<code>nc -zv example.com 80</code>

5 EXAMPLE SCENARIO WALKTHROUGH

"Your application cannot connect to the database. How would you troubleshoot this?"

- 1 Check DNS - Can you resolve the database hostname?
- 2 Check route - Is the network path correct?
- 3 Check firewall - Is the traffic allowed?
- 4 Check port - Is the database port open?
- 5 Check process - Is the database service listening?
- 6 Check application - Is the database actually healthy?
- 7 Verify - Test the connection again after fixes.

4 COMMON ERRORS AND WHAT THEY MEAN

Connection refused	Host is reachable but nothing is listening on the port.
Connection timeout	No response from the host. Could be firewall, route, or host down.
DNS resolution failure	Domain name cannot be resolved to an IP address.
No route to host	Network routing issue.
Port closed	Port is not open on the target host.
Connection reset	Connection was established but closed by the remote host.

6 JUNIOR ENGINEER TIPS

- ✓ Start with the basics (DNS, network, port).
- ✓ Do not assume it is the application.
- ✓ Use the right tool for each layer.
- ✓ Explain your thinking as you troubleshoot.
- ✓ Mention what you would check next if the issue persists.
- ✓ Consider security rules and cloud network components (e.g. Security Groups, NACLs, firewalls).

Think
Investigate
Solve
Explain



Do not rush straight to a fix.

Understand the problem, gather evidence, and think through the layers.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

ANSWERING KUBERNETES TROUBLESHOOTING SCENARIOS



Same Skills
Bigger Opportunities

A practical guide to diagnose and solve common Kubernetes problems in technical interviews and in real production environments.

Think Troubleshoot
Explain
Succeed

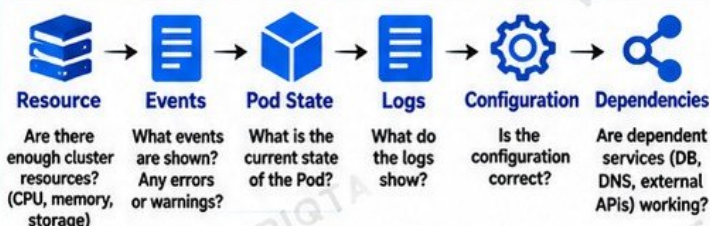
1 COMMON INTERVIEW SITUATIONS

- Pod Pending
- CrashLoopBackOff
- ImagePullBackOff
- OOMKilled
- Failed readiness probe
- Service cannot reach Pods
- Deployment rollout fails

"Interviewers want to see how you think, troubleshoot, and use the right tools — not just the final answer."

2 REASONING SEQUENCE

Follow this sequence to narrow down the issue.



"Move step by step, don't jump to conclusions."

3 USEFUL KUBECTL COMMANDS

Command	What It Does	Example Usage
<code>kubectl get</code>	List resources	<code>kubectl get pods</code> <code>kubectl get svc</code>
<code>kubectl describe</code>	Show detailed information	<code>kubectl describe pod <pod-name></code>
<code>kubectl logs</code>	View container logs	<code>kubectl logs <pod-name> -c <container></code>
<code>kubectl get events</code>	View cluster events	<code>kubectl get events --sort-by=.lastTimestamp</code>
<code>kubectl rollout status</code>	Check deployment rollout status	<code>kubectl rollout status deployment/<name></code>

4 COMMON ISSUES, CAUSES AND WHAT TO CHECK

Scenario	Possible Causes	What to Check
Pod Pending	Insufficient resources, node selector, taints, PVC issues	Describe pod, check events, node resources, storage claims
CrashLoopBackOff	App error, misconfig, dependency failure	Logs, environment variables, config, dependent services
ImagePullBackOff	Wrong image name, registry auth, network issue	Describe pod, events, image name, imagePullSecrets, registry access
OOMKilled	Container using too much memory	Describe pod, check resource limits, container logs
Failed readiness probe	App not ready, wrong probe configuration	Describe pod, check probe settings, application logs
Service cannot reach Pods	Wrong labels, selectors, network policy	Check service, endpoints, network policies, pod labels
Deployment rollout fails	Image error, failing pods, misconfiguration	<code>kubectl rollout status</code> , describe deployment, logs, replica sets

5 EXAMPLE TROUBLESHOOTING WORKFLOW

Example: Pod is in CrashLoopBackOff

- 1 Check the pod status – `kubectl get pods`
- 2 Describe the pod – `kubectl describe pod`
- 3 Check events – `kubectl get events`
- 4 View logs – `kubectl logs <pod>`
- 5 Check configuration – env, configmaps, secrets
- 6 Verify dependencies – database, services, external APIs
- 7 Make a fix – update config, image, or dependency
- 8 Verify – pod is running and healthy

6 JUNIOR ENGINEER TIPS

- ✓ Start with the basics (status, events, logs).
- ✓ Check recent changes (deployment, config, image).
- ✓ Use `kubectl describe` to get detailed clues.
- ✓ Look at logs before making assumptions.
- ✓ Understand the application and its dependencies.
- ✓ Explain your thinking clearly during the interview.
- ✓ Don't just restart the pod — find the root cause.

It's not just about the commands. It's about how you think.



REMEMBER

Troubleshooting is a process. Gather evidence, think through the problem, and explain your approach.



ANSWERING CLOUD INFRASTRUCTURE SCENARIOS



SAME CONCEPTS
ACROSS CLOUD PROVIDERS

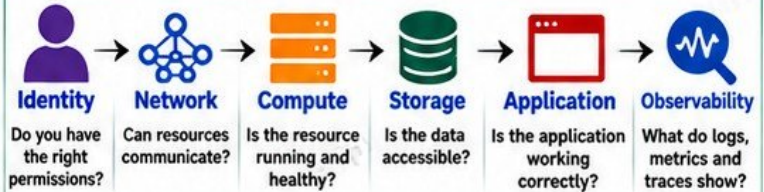
A practical guide to troubleshoot and solve common cloud infrastructure problems in technical interviews and in real production environments.

1 COMMON INTERVIEW SCENARIOS

- ❗ VM is unreachable
- ❗ Application is unavailable
- ❗ Load balancer reports unhealthy targets
- ❗ Storage is inaccessible
- ❗ Cloud bill suddenly increases
- ❗ Instance was accidentally terminated
- ❗ Resource cannot access another resource
- ❗ Availability Zone failure

2 INVESTIGATION MODEL

Inspect each layer in this order.



“Start broad, then go deeper.
Check each layer before moving to the next.”

3 SCENARIO OVERVIEW AND WHAT TO CHECK

Scenario	Key Things to Check
VM is unreachable	Security groups, network ACLs, route tables, public/private IP, instance state, SSH/RDP access
Application is unavailable	Compute health, load balancer, target group, service health, logs, recent changes
Load balancer reports unhealthy targets	Target health checks, security groups, instance status, application logs, health endpoint
Storage is inaccessible	Permissions (IAM), network access, mount configuration, encryption, storage service health
Cloud bill suddenly increases	Recently created resources, scaling, unused resources, data transfer, logging, misconfiguration
Instance was accidentally terminated	Check recovery options (AMI, snapshots, auto scaling), termination protection, recent activities
Resource cannot access another resource	IAM permissions, security groups, network routing, VPC peering, private endpoints
Availability Zone failure	Multi-AZ setup, failover, health checks, routing, replication, resilience strategy

4 USEFUL TOOLS AND COMMANDS

Tool / Command	What It Does	Example (AWS)
<code>aws cli</code>	Interact with cloud resources	<code>aws ec2 describe-instances</code>
<code>cloud console</code>	Visual management	EC2, VPC, RDS, S3, etc.
<code>curl</code>	Test application endpoints	<code>curl https://example.com</code>
<code>ping</code>	Test network connectivity (where allowed)	<code>ping <ip-address></code>
<code>traceroute</code>	Trace network path	<code>traceroute <ip-address></code>
<code>nslookup / dig</code>	DNS resolution	<code>nslookup example.com</code>
<code>telnet / nc</code>	Test port connectivity	<code>nc -zv example.com 80</code>
<code>aws logs</code>	Check logs (CloudWatch)	<code>aws logs get-log-events</code>
<code>aws s3</code>	Work with S3 storage	<code>aws s3 ls</code>
<code>aws rds</code>	Check database status	<code>aws rds describe-db-instances</code>
<code>aws elbv2</code>	Check load balancer health	<code>aws elbv2 describe-target-health</code>
<code>aws sts</code>	Check identity and permissions	<code>aws sts get-caller-identity</code>

5 EXAMPLE SCENARIO WALKTHROUGH

Example: Application is unavailable

- 1 Check identity – do you have the right permissions?
- 2 Check network – are security groups, routes and DNS correct?
- 3 Check compute – are the instances/pods running?
- 4 Check storage – is the database or storage accessible?
- 5 Check application – are the services and health checks working?
- 6 Check observability – what do logs and metrics show?
- 7 Identify root cause and apply the fix.
- 8 Verify the application is fully recovered.

6 JUNIOR ENGINEER TIPS

- ✓ Always start by understanding the scope and impact.
- ✓ Check your permissions first.
- ✓ Use the investigation model (Identity → Network → Compute → Storage → Application → Observability).
- ✓ Look for recent changes.
- ✓ Don't assume it's the application – check the infrastructure.
- ✓ Explain your thinking during the interview.
- ✓ Mention high availability and disaster recovery where relevant.
- ✓ Be aware of cost implications.
- ✓ It's okay to say “I would check...” instead of giving a final answer immediately.

Think
Investigate
Solve
Explain
Improve



REMEMBER

In the cloud, most problems are not solved by one command. Investigate, understand, and explain your approach.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Same
Skills
Bigger
Opportunities

ANSWERING CI/CD PIPELINE FAILURE SCENARIOS

A practical guide to troubleshoot and solve common CI/CD pipeline problems in technical interviews and in real production environments.

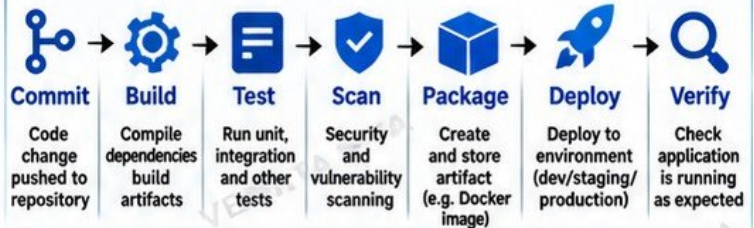
1 COMMON INTERVIEW SCENARIOS

- ❗ Build suddenly fails
- ❗ Tests fail
- ❗ Deployment stage fails
- ❗ Credentials expire
- ❗ Artifact cannot be found
- ❗ Runner is unavailable
- ❗ Production deployment fails after CI succeeds

"Interviewers want to see how you think, investigate, and isolate the problem – not just give a quick fix."

2 CI/CD PIPELINE FLOW (REASONING)

Follow the pipeline stages to find where it stopped.



Find exactly where the pipeline stopped before proposing a fix.

3 COMMON CAUSES AND WHAT TO CHECK

Scenario	Possible Causes	What to Check
Build fails	Dependency issues, wrong version, build configuration	Build logs, dependencies, build file (e.g. pom.xml, build.gradle, Dockerfile)
Tests fail	Failing test cases, environment differences	Test logs, test reports, configuration, test data
Deployment stage fails	Wrong config, missing permissions, resource limits	Deployment logs, manifests, permissions, environment settings
Credentials expire	Expired tokens, keys, service accounts	Secrets, token expiry, CI/CD variables, IAM permissions
Artifact cannot be found	Build did not produce artifact, wrong path, repo issue	Build logs, artifact repo (e.g. Nexus, Artifactory, ECR)
Runner is unavailable	No available runners, runner offline, capacity issue	Runner status, logs, capacity, configuration
Production deployment fails after CI succeeds	Environment-specific issue, missing config, failed health checks, manual approval missing	Deployment logs, cluster/service status, configuration, health checks

5 EXAMPLE SCENARIO WALKTHROUGH

Example: Deployment stage fails

- 1 Check which stage failed – look at the pipeline logs.
- 2 Review the error message – what is the exact failure?
- 3 Check recent changes – code, configuration, environment.
- 4 Verify credentials and permissions.
- 5 Check the target environment – is the cluster/service healthy?
- 6 Fix the issue and re-run the pipeline.

4 USEFUL TOOLS AND COMMANDS

Tool / Command	What It Does	Example Usage
View pipeline logs (General)	View detailed logs for each stage	Check in your CI/CD platform (e.g. GitHub Actions, GitLab CI, Jenkins, Azure DevOps)
Check build logs	See why the build failed	Platform UI or CLI
Check test results	View test reports	Platform UI or artifacts
kubectl logs	View application logs (after deployment)	<code>kubectl logs <pod-name></code>
kubectl get pods	Check pod status	<code>kubectl get pods</code>
kubectl describe	Get detailed information	<code>kubectl describe pod <pod></code>
Check runner status	Verify available runners	Platform settings (e.g. GitLab Runners, GitHub Actions)
Check artifacts	List and download artifacts	Platform UI or CLI
Check deployment status	Verify rollout status	<code>kubectl rollout status deployment/<name></code>

6 JUNIOR ENGINEER TIPS

- ✓ Start by identifying which stage failed.
- ✓ Read the logs – they tell you what happened.
- ✓ Check recent changes (code, config, secrets, environment).
- ✓ Verify permissions and credentials.
- ✓ Understand the difference between build, test and deploy issues.
- ✓ Don't assume it's the code – it could be environment or config.
- ✓ Explain your thinking clearly during the interview.
- ✓ Find the root cause, not just a temporary fix.

Think
Investigate
Solve
Explain
Improve



REMEMBER

Always find exactly where the pipeline stopped before proposing a fix. Do not guess.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Same
Skills
Bigger
Opportunities

ANSWERING DEPLOYMENT AND RELEASE FAILURE SCENARIOS

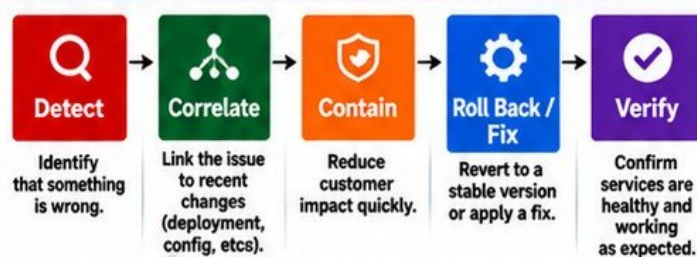
A practical guide to troubleshoot and solve deployment and release problems in technical interviews and in real production environments.

Think
Investigate
Solve
Explain
Improve

1 COMMON INTERVIEW SCENARIOS

- ! New release causes errors
- ! Error rate increases after deployment
- ! Health checks fail
- ! Some users receive the old version
- ! Database migration causes problems
- ! Configuration is incorrect
- ! Deployment must be rolled back

2 DEPLOYMENT TROUBLESHOOTING FLOW



Follow this flow in order. Don't skip steps.

3 WHAT TO CHECK AT EACH STAGE

Stage	What to Check	Example Questions
Detect	Alerts, error rates, health checks, user reports	What is failing? Who is affected? When did it start?
Correlate	Recent deployments, config changes, database changes	What changed recently? Is the timing related to the issue?
Contain	Roll back, feature flag, scale, isolate	How can I reduce impact quickly?
Roll Back / Fix	Revert to previous version or apply fix	Is rollback possible? What is the fix? Does it need a config change?
Verify	Check health, logs, metrics, user access	Is the service fully restored? Are error rates back to normal?

4 WHEN ROLLBACK IS SAFER THAN DEBUGGING

! ROLLBACK IS OFTEN THE SAFER CHOICE WHEN:

- ✓ The application is causing customer-facing downtime.
- ✓ Error rate or latency is high and increasing.
- ✓ The issue affects critical business functions.
- ✓ You do not yet know the root cause.
- ✓ A previous version is stable and known to work.
- ✓ Data corruption or further damage is a risk.
- ✓ It will take too long to debug in production.
- ✓ The change is not easily reversible by a quick fix.

"In production, restoring service quickly is usually more important than finding the root cause immediately. You can debug after the system is stable."

5 EXAMPLE SCENARIO WALKTHROUGH

Example: New release causes high error rate

- 1 Detect - Alerts show 500 errors after deployment.
- 2 Correlate - Check recent deployment and config changes.
- 3 Contain - Roll back to the previous stable version.
- 4 Verify - Confirm error rate is back to normal.
- 5 Investigate - Review logs and identify the root cause.
- 6 Fix - Update code/configuration.
- 7 Prevent - Improve testing, monitoring, or rollout strategy.

6 COMMANDS AND TOOLS (EXAMPLES)

Area	Command / Tool	What It Does
Kubernetes	<code>kubectl rollout status</code>	Check deployment status
Kubernetes	<code>kubectl rollout undo</code>	Roll back to previous version
Logs	<code>kubectl logs <pod></code>	View application logs
Monitoring	Grafana / Prometheus	Check metrics and error rates
CI/CD	GitHub Actions / GitLab CI	Check pipeline and deployment logs
Cloud	AWS / Azure / GCP Console	Check service health and events
Application	<code>curl / health endpoint</code>	Verify service response

7 JUNIOR ENGINEER TIPS

- ✓ Always check what changed recently.
- ✓ Look at logs, metrics, and health checks before making assumptions.
- ✓ Consider rollback when customer impact is high.
- ✓ Communicate clearly about the impact and your actions.
- ✓ Verify the system is stable after rollback or fix.
- ✓ Document what happened and propose improvements.
- ✓ Understand the deployment process for your environment.
- ✓ Practice explaining your thought process clearly in interviews.

It's not
just a deployment.
It's about
reliable systems.



REMEMBER Restoring service quickly, safely, and confidently is a key part of an engineer's job.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

DEVOPS | CLOUD | SRE | PLATFORM | INFRASTRUCTURE

Practice
Build
Grow
with VERIQTA

ANSWERING MONITORING AND OBSERVABILITY SCENARIOS

A practical guide to troubleshoot and solve monitoring and observability problems in technical interviews and in real production environments.

Think
Observe
Analyse
Solve
Improve

1 COMMON INTERVIEW SCENARIOS

- ❗ CPU alert fires
- ❗ Latency suddenly increases
- ❗ Error rate rises
- ❗ Alert fires but customers report no problem
- ❗ Dashboard looks healthy but users report failures
- ❗ Logs show repeated errors

"Interviewers want to see how you use metrics, logs and traces to investigate, and how you think about signal vs noise."

2 THE OBSERVABILITY PILLARS

METRICS



- Tell you something is happening
- Numerical data over time
- Used for alerting and trends
- Examples: CPU, memory, request rate, latency, error rate

LOGS



- Provide event detail
- Show what happened
- Help you understand the cause
- Useful for debugging
- Examples: application logs, system logs, container logs

TRACES



- Show request paths
- Help you understand how services interact
- Identify where latency or errors occur
- Useful in distributed systems
- Examples: end-to-end request flow, service dependencies

"Metrics tell you something is happening.

Logs provide event detail. Traces show request paths."

3 INVESTIGATION FLOW



- 1. Check the alert**
What fired?
What is the symptom?
- 2. Review metrics**
Look at trends and compare with normal baseline.
- 3. Examine logs**
Find related events and error messages.
- 4. Check traces**
Follow the request path and find where the issue occurs.
- 5. Identify root cause**
Correlate all three data sources before taking action.
- 6. Resolve and verify**
Fix the issue and confirm metrics, logs and traces are normal.

Use metrics to detect. Use logs to investigate. Use traces to understand.

4 SIGNAL VS NOISE



SIGNAL
(Important)

- ✓ Real issue affecting users
- ✓ Consistent pattern
- ✓ Correlates across multiple sources (metrics, logs, traces)
- ✓ Matches recent changes
- ✓ Requires investigation



NOISE
(Likely not important)

- ✓ Single, short spike
- ✓ No user impact
- ✓ Not consistent
- ✓ No correlation with other data
- ✓ Known and expected behaviour
- ✓ Can often be ignored or silenced

"Always validate an alert. Not every alert means a real problem. Look for patterns and corroborate with multiple data sources."

5 EXAMPLE SCENARIOS AND HOW TO APPROACH THEM

Scenario	What to Check	Example Questions
CPU alert fires	CPU metrics, running processes, recent deployments, logs	Is this a real spike? What changed recently? Which process is using CPU?
Latency increases	Latency metrics, traces, downstream services, logs	Where is the latency coming from? Which service is slow?
Error rate rises	Error metrics, logs, traces, recent changes	What errors are occurring? Is it a new deployment?
Alert fires but no user impact	Check metrics in detail, time range, known issues	Is this a false positive? Is it a known maintenance or batch job?
Dashboard healthy but users report issues	Check the right metrics, logs, traces, and different time ranges	Am I looking at the correct service? Is there a regional or user-specific issue?
Logs show repeated errors	Investigate log messages, correlate with metrics and traces	What is causing the errors? Is it affecting users? Did it start recently?

6 JUNIOR ENGINEER TIPS

- ✓ Understand what each alert means.
- ✓ Always check multiple data sources (metrics, logs, traces).
- ✓ Look at the time range and compare with normal behaviour.
- ✓ Check for recent deployments or configuration changes.
- ✓ Consider whether the issue is user-facing.
- ✓ Do not blindly trust a single dashboard.
- ✓ Learn the normal patterns for your systems.
- ✓ Communicate clearly about what you found.
- ✓ Escalate when the impact is high or unclear.
- ✓ Document your findings and the resolution.

It's not just data. It's real users.

7 COMMON TOOLS (EXAMPLES)



8 KEY TAKEAWAY



"Use metrics, logs and traces together, focus on real user impact, filter out the noise, and think end-to-end."



REMEMBER

Observability is not just about tools. It's about asking the right questions, finding the real problem, and helping users get back to a healthy system.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

DEVOPS | CLOUD | SRE | PLATFORM | INFRASTRUCTURE

Practice
Build
Grow
with VERIQTA

ANSWERING INCIDENT AND ON-CALL SCENARIOS

Stay calm. Think clearly. Restore service. Communicate. Learn.



EXAMPLE SCENARIO

"It is 2 AM and production is down. What do you do?"



KEY MINDSET

- Focus on restoring service first.
- Be methodical, not panicked.
- Communicate clearly.
- Make safe and reversible changes.
- Deep root-cause analysis comes after stabilization.

STEP-BY-STEP RESPONSE

1	Acknowledge the alert	Confirm you received the alert and start looking at it.
2	Confirm the incident	Verify that there is a real issue, not a false alarm.
3	Assess customer impact	Determine who is affected (all users or some) and what is impacted.
4	Establish severity	Define the severity level (e.g. SEV1, SEV2) based on business impact.
5	Gather evidence	Check metrics, logs, system status, and error messages.
6	Check recent changes	Look at deployments, configuration changes, infrastructure changes.
7	Mitigate quickly	Apply a safe fix or rollback to restore service as fast as possible.
8	Communicate status	Keep stakeholders updated with clear and regular status updates.
9	Escalate when necessary	Involve the right team members if you need more expertise or access.
10	Verify recovery	Confirm the service is stable and working as expected.
11	Preserve evidence	Keep logs, metrics, and records for later analysis.
12	Document the timeline	Write a clear timeline of what happened and what actions were taken.
13	Follow with a post-incident review	Identify the root cause and action items to prevent it from happening again.



RESTORATION FIRST

Get the service working before deep investigation when customers are impacted.



BE TRANSPARENT

Communicate early, communicate clearly, and avoid silence.



ALWAYS LEARN

Every incident is an opportunity to improve systems, processes, and your engineering skills.



JUNIOR ENGINEER TIP

Do not rush straight to a fix. Take time to understand the problem, gather evidence, and consider the impact of your actions. A calm, structured approach builds trust.



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

SECURITY SCENARIOS IN TECHNICAL INTERVIEWS

Security scenarios test your ability to detect risks, respond correctly, and protect systems, data, and users in real-world situations.

1 COMMON SECURITY SCENARIOS



Exposed credential

API keys, passwords or tokens exposed.



Secret committed to Git

Sensitive data in a repository.



Suspicious login

Unusual or unauthorized access attempts.



Overly permissive IAM role

Too many permissions (Violation of least privilege).



Publicly exposed storage

S3 bucket or storage accessible to anyone.



Unexpected network traffic

Suspicious or unusual inbound/outbound traffic.



Compromised workload

Possible malware, unauthorized access or data exfiltration.



Other scenarios

e.g. vulnerability in a dependency, misconfiguration, etc.

2 RESPONSE MODEL

A structured approach you can use for any security incident.



CONTAIN

Stop the threat and limit further damage.



REVOKE / ROTATE

Revoke or rotate compromised credentials or access.



INVESTIGATE

Find out what happened, how, and the full scope.



RECOVER

Restore systems and ensure they are secure.



VERIFY

Confirm the threat is removed and systems are working normally.



PREVENT

Implement controls to reduce the chance of recurrence.

3 IMPORTANT LESSON



Never expose credentials or destroy evidence while troubleshooting.

You could make the situation worse and lose important forensic information.

4 JUNIOR ENGINEER TIPS



- ✓ Stay calm and think before acting.
- ✓ Follow the response model.
- ✓ Focus on the impact and scope.
- ✓ Communicate clearly.
- ✓ Ask for help when needed.
- ✓ Explain your reasoning during the interview.



"Security is everyone's responsibility. A good engineer not only fixes problems, but also helps prevent them."

Secure
Build
Better



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Same
Engineers
Stronger
Tomorrow

HOW TO HANDLE “YOU MADE THE PRODUCTION MISTAKE”

Mistakes
Happen
Real Engineers
Learn
Grow

Own it. Fix it. Learn from it. That's what great engineers do.



EXAMPLE SCENARIO



“You deployed a change and production went down. What would you do?”



WHAT INTERVIEWERS ARE TESTING



- Professional judgment
- Accountability
- Problem-solving ability
- Communication skills
- How you handle pressure
- Whether you can learn from mistakes

1

STEP-BY-STEP RESPONSE

1		Take ownership	Be honest and accept responsibility. Do not blame others.
2		Focus on restoration	Your first priority is to get the service back up and minimize customer impact.
3		Do not hide the change	Be transparent about what you changed, when, and why.
4		Notify the appropriate people	Inform your team, manager, and stakeholders early.
5		Determine whether rollback is safe	Assess the risks of rolling back (e.g. data loss, migrations, etc.).
6		Verify service restoration	Confirm the service is working as expected and monitor closely.
7		Document what happened	Record the timeline, what changed, what went wrong, and how it was fixed.
8		Identify contributing controls	Look at gaps in testing, review, automation, monitoring, or processes.
9		Improve for the future	Recommend and implement changes to prevent a similar issue.



KEY MINDSET

- Be accountable
- Stay calm
- Communicate clearly
- Focus on solutions
- Turn mistakes into learning



IMPORTANT LESSON

This page is important because it tests **professional judgment** as much as technical skill.



RESULT

- Restored service
- Maintained trust
- Demonstrated maturity
- Learned from the mistake
- Became a stronger engineer



“A great engineer is not someone who never makes mistakes, but someone who handles them well and helps prevent them in the future.”

Better
Engineers
Brighter
Futures



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

It's OK
Not to Know
What Matters
is How You
Handle It

HOW TO ANSWER WHEN YOU DO NOT KNOW

Same
Engineers
Brighter
Futures

You do not need to know everything. You need to think like an engineer.



WHY INTERVIEWERS ASK QUESTIONS YOU MAY NOT KNOW



- Real-world problems are complex
- No one knows everything
- They want to see your thinking process
- They are testing how you handle uncertainty
- It shows if you can learn, adapt, and find solutions



DO NOT BLUFF



- Do not pretend you know
- Do not give random answers
- Do not make up tools, commands, or concepts
- Do not oversimplify a complex problem
- Losing trust is worse than saying "I don't know"



THE RIGHT WAY TO ANSWER

1



STATE WHAT YOU KNOW

"Based on what I know, ..."
Start with the relevant knowledge you have.

2



IDENTIFY WHAT YOU DO NOT KNOW

"I'm not certain about ..."
Be honest about the gaps in your knowledge.

3



EXPLAIN YOUR ASSUMPTIONS

"I'm assuming ..."
State any assumptions clearly so the interviewer understands your context.

4



DESCRIBE WHAT YOU WOULD INSPECT

"I would start by checking ..."
Explain what information you would gather and why.

5



EXPLAIN HOW YOU WOULD FIND THE MISSING INFORMATION

"I would look at the documentation,
logs, or consult ..."
Show how you would learn or find the answer.

6



USE DOCUMENTATION APPROPRIATELY

"I would check the official documentation ..."
Good engineers use documentation. It shows professionalism.

7



ASK ANOTHER ENGINEER WHEN RISK REQUIRES IT

"I would reach out to a more experienced
engineer ..."
Know when to collaborate and escalate.

8



NEVER EXPERIMENT BLINDLY IN PRODUCTION

"I would avoid making changes in production
without understanding the impact."
Safety and risk awareness are critical.



EXAMPLE ANSWER

"I haven't encountered this exact failure before. I would start by checking the logs and system status to understand the symptoms. I would also review the official documentation and look for similar issues. If needed, I would ask a more experienced engineer for guidance before taking any action, especially in production."



WHAT THIS SHOWS THE INTERVIEWER

- Honesty
- Logical thinking
- Problem-solving approach
- Willingness to learn
- Good judgment
- Teamwork mindset
- Awareness of production risk
- Professional communication



KEY TAKEAWAY

**Not knowing
something is normal.**
How you respond
shows the difference
between a junior
and a professional
engineer.



"Great engineers are not defined by what they know, but by how they find out what they don't know."



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Think
Explain
Solve
Grow

WEAK ANSWERS vs STRONG ENGINEERING ANSWERS

It's not just what you say, but how you think, explain, and justify your approach.

SCENARIO	✗ WEAK ANSWER	✓ STRONG ENGINEERING ANSWER
 Server is slow or unresponsive	"I would restart the server."	"I would first determine whether the issue affects one instance or the entire service, then inspect resource utilization, service state, logs, and recent changes before deciding whether a restart is appropriate."
 Application errors	"Check logs."	"I would check the relevant application logs, system logs, and any recent deployment logs to identify the error, look for patterns, and correlate with metrics and recent changes."
 Deployment issue	"Rollback."	"I would assess the impact, check what changed, review deployment status and error details, and then decide if a rollback is the safest option."
 Performance issue	"Scale up."	"I would first identify the actual bottleneck (CPU, memory, I/O, database, etc.) using metrics and logs, then decide whether scaling is the right solution or if there is a configuration or code issue that needs to be fixed."
 Use of Kubernetes	"Use Kubernetes."	"I would use the appropriate Kubernetes resource depending on the problem, such as a Deployment for stateless applications, a StatefulSet for stateful apps, a ConfigMap for configuration, or a Service for networking, and explain why it fits the use case."
 Pod is crashing	"Restart the Pod."	"I would first check the Pod status, logs, and events to understand why it is crashing (e.g. image issue, config error, insufficient resources), then fix the root cause before restarting."

⚠️ AVOID GENERIC ANSWERS

- They show a lack of depth.
- They make you seem inexperienced.
- They do not demonstrate engineering judgment.

✓ SHOW YOUR THINKING

- Explain your reasoning.
- Mention what you would check and why.
- Consider alternatives.
- Show awareness of impact and risk.

⚙️ THINK LIKE A PROFESSIONAL

- Be methodical.
- Do not jump to a fix.
- Consider the bigger picture.
- Explain trade-offs.
- Communicate clearly.

 "Strong engineers don't just give answers, they show their thinking."

Learn
Practice
Build
Better



COMPLETE MOCK SCENARIO INTERVIEW

Think like an engineer. Walk through the problem. Communicate clearly.

It's not just
about the answer,
it's about your
thinking.



SCENARIO



Customers report
intermittent 500 errors
shortly after a production
deployment.



WHAT THE INTERVIEWER WANTS TO SEE

- ✓ How you think and troubleshoot
- ✓ Your ability to ask the right questions
- ✓ Your technical and communication skills
- ✓ Your decision-making under pressure
- ✓ Your awareness of production risk
- ✓ How you handle uncertainty



INTERVIEWER FOLLOW-UPS

- 1 "What if rollback fails?"
- 2 "What if only 20% of users are affected?"
- 3 "What if the logs show nothing?"
- 4 "What would make you escalate?"
- 5 "How would you confirm it's fixed?"
- 6 "What if it happens again after the fix?"
- 7 "What if you cannot reproduce the issue?"
- 8 "How would you prevent this in the future?"



JUNIOR ENGINEER TIP

It's OK if you've never seen this exact issue before. What matters is how you approach it, how you think, and how you communicate your reasoning.



STEP-BY-STEP APPROACH

1



CLARIFY

- Ask questions to understand the problem fully.
- What exactly is happening?
- When did it start?
- Who is affected?
- What changed recently?

2



ASSESS IMPACT

- How many users are affected?
- Is it all users or a percentage?
- Is it a full outage or intermittent?
- What is the business impact?

3



CHECK RECENT CHANGES

- Look at recent deployments.
- Any configuration changes?
- Dependency or infrastructure changes?

4



METRICS + LOGS + STATE

- Check application logs.
- Check infrastructure metrics.
- Check deployment status.
- Look at dependencies (database, external APIs, etc.).

5



FORM HYPOTHESES

- List possible causes.
- Prioritize the most likely causes.

6



TEST HYPOTHESES

- Gather more evidence.
- Test one hypothesis at a time.
- Eliminate possibilities.

7



MITIGATE / ROLL BACK

- Apply a safe fix or rollback if necessary.
- Consider feature flag or partial rollback.

8



VERIFY RECOVERY

- Confirm the error rate is back to normal.
- Monitor for a period of time.

9



ROOT CAUSE

- Identify the real cause.
- Explain what happened and why.

10



PREVENTION

- Suggest improvements.
- Testing, monitoring, deployment process, or architecture changes.



EXAMPLE RESPONSE (SUMMARY)

"I would start by confirming the scope of the issue, checking recent deployments, reviewing application logs and metrics, and looking at dependency health. If the issue started after a deployment, I would compare the current version with the previous one and consider a rollback if necessary. Once the service is stable, I would investigate the root cause and recommend changes to prevent it from happening again."



KEY POINTS TO REMEMBER

- ✓ Be structured and methodical.
- ✓ Think out loud.
- ✓ Ask questions before assuming.
- ✓ Use evidence, not guesses.
- ✓ Communicate clearly.
- ✓ Consider customer impact.
- ✓ Know when to roll back.
- ✓ Show you can learn from the incident.



COMMON PITFALLS

- ✗ Jumping to a fix without investigation.
- ✗ Ignoring recent changes.
- ✗ Not considering impact.
- ✗ Giving vague answers.
- ✗ Blaming others.
- ✗ Forgetting to verify the fix.
- ✗ Not thinking about prevention.
- ✗ Being overconfident.



FINAL THOUGHT

"Interviewers are not looking for perfection. They are looking for engineers who can think, solve problems, communicate, and make good decisions in real situations."

Better
Engineers
Brighter
Futures



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta

Think
Like the
Engineer

FINAL INTERVIEW SIMULATION

PRACTICE REAL SCENARIOS. BUILD REAL CONFIDENCE.

These scenarios cover real-world situations across infrastructure, cloud, containers, CI/CD, databases, security, and production. Use the same structured thinking for every problem.

Learn
Practice
Grow

1 SERVER PERFORMANCE



Production server reaches 100% CPU.

2 KUBERNETES



Kubernetes Pod enters CrashLoopBackOff.

3 NETWORKING / APP



Users cannot reach an application.

4 CI/CD



CI/CD deployment fails.

5 DATABASE



Database latency suddenly increases.

6 SECURITY



A secret appears in a public repository.

7 DEPLOYMENT



Production fails immediately after your deployment.

8 MONITORING



Monitoring says healthy, customers say otherwise.

YOUR ANSWER FRAMEWORK (USE THIS FOR EVERY SCENARIO)



- 1 What would you clarify?
- 2 What is the impact?
- 3 What evidence would you gather?
- 4 What are your first hypotheses?
- 5 How would you test them?
- 6 What would you do to mitigate the problem?
- 7 How would you verify recovery?
- 8 How would you reduce the chance of recurrence?

FINAL MENTAL MODEL

✗ DON'T

Guess
↓
Command
↓
Random Fix

✓ DO

Clarify
↓
Assess Impact
↓
Gather Evidence
↓
Form Hypotheses
↓
Test
↓
Mitigate / Fix
↓
Verify
↓
Prevent



INTERVIEW TIPS

- ✓ Think out loud.
- ✓ Ask questions.
- ✓ Be structured.
- ✓ Explain your reasoning.
- ✓ Consider trade-offs.
- ✓ Communicate clearly.



KEY TAKEAWAY

It's not about knowing every answer. It's about how you think, solve problems, and communicate like a real engineer.

“Interviews don't test perfection. They test how you think in the real world.”

– VERIQTA



@veriqta



veriqta



veriqta



veriqta



@veriqta



@veriqta